

Chương 8:

MẠCH LOGIC

ThS. GVC Tô Oai Hùng

1

1

Nội Dung

1. Giới thiệu.
2. Đại số Boolean và các cổng logic.
3. Mạch tổ hợp.
4. Mạch tuần tự.
5. Thanh ghi.

ThS. GVC Tô Oai Hùng

2

2

8.1 Giới Thiệu

- ❖ Các hoạt động của máy tính số được dựa trên việc lưu trữ và xử lý dữ liệu nhị phân.
- ❖ Chúng ta giả định rằng các phần tử lưu trữ có thể tồn tại ở một trong hai trạng thái ổn định.
- ❖ Các mạch có thể hoạt động trên dữ liệu nhị phân dưới sự điều khiển của tín hiệu điều khiển để thực hiện các chức năng máy tính khác nhau.
- ❖ Trong chương này, chúng ta xem xét cách mà các phần tử lưu trữ và mạch có thể được thực hiện trong logic số, đặc biệt với các mạch tổ hợp và tuần tự.

ThS. GVC Tô Oai Hùng

3

3

8.1 Giới Thiệu

- ❖ Chương này sẽ xem xét các thành phần cơ bản mà từ đó tạo nên tất cả máy tính.
- ❖ Bắt đầu bằng đại số Boolean, nền tảng toán học của logic số.
- ❖ Tiếp theo là giới thiệu về một số cổng logic. Cuối cùng là thiết kế một số mạch tổ hợp và mạch tuần tự.

ThS. GVC Tô Oai Hùng

4

4

8.2 Đại Số Boolean và Các Cổng Logic

1. Đại số Boolean.
2. Các cổng logic.

ThS. GVC Tô Oai Hùng

5

5

8.2.1 Đại Số Boolean

- ❖ Mạch kỹ thuật số trong máy tính và các hệ thống số khác được thiết kế sử dụng lý thuyết đại số Boolean (*Boolean algebra*).
- ❖ Đại số Boolean được phát minh bởi nhà toán học người Anh George Boole vào năm 1854.
- ❖ Nó là công cụ tiện lợi trong hai lĩnh vực:
 - Phân tích: Tiết kiệm thời gian khi mô tả chức năng của mạch số.
 - Thiết kế: Đơn giản trong việc hiện thực chức năng muốn thiết kế.
- ❖ Trong đại số Boolean, các phép toán thực hiện trên các biến nhị phân (0/1).

ThS. GVC Tô Oai Hùng

6

6

Đại Số Boolean

- ❖ Đại số Boolean bao gồm một số luật và chỉ có ba phép toán luận lý: AND, OR và NOT.
- ❖ Chúng được ký hiệu như sau:

$$A \text{ AND } B = A \cdot B = AB$$

$$A \text{ OR } B = A + B$$

$$\text{NOT } A = \bar{A} = A'$$
- ❖ Bảng 2.1 thể hiện kết quả một số phép toán, gọi là bảng sự thật hay bảng chân trị (*truth table*). Trong đó:
 - Phép toán XOR: Kết quả là 1 khi chỉ có một trong hai toán hạng là 1.
 - Phép toán NAND: Là NOT của AND.
 - Phép toán NOR: Là NOT của OR.

ThS. GVC Tô Oai Hùng

7

7

Đại Số Boolean

P	Q	NOT P ($\bar{P}$)	P AND Q ($P \cdot Q$)	P OR Q ($P + Q$)	P NAND Q ($\overline{P \cdot Q}$)	P NOR Q ($\overline{P + Q}$)	P XOR Q ($P \oplus Q$)
0	0	1	0	0	1	1	0
0	1	1	0	1	1	0	1
1	0	0	0	1	1	0	1
1	1	0	1	1	0	0	0

Bảng 2.1: Các phép toán Boolean cho hai biến P và Q.

ThS. GVC Tô Oai Hùng

8

8

Đại Số Boolean

❖ Các luật của đại số Boolean:

1. Luật giao hoán (*Commutative law*):

$$A \cdot B = B \cdot A$$

$$A + B = B + A$$

2. Luật phân phối (*Distributive law*):

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

3. Phần tử đồng nhất (*Identity element*):

$$1 \cdot A = A$$

$$0 + A = A$$

4. Phần tử bù/nghịch đảo (*Inverse element*):

$$A \cdot \bar{A} = 0$$

$$A + \bar{A} = 1$$

ThS. GVC Tô Oai Hùng

9

9

Đại Số Boolean

5. Luật trội/nuốt/null (*Null law*) :

$$0 \cdot A = 0$$

$$1 + A = 1$$

6. Luật lũy đẳng (*Idempotent law*):

$$A \cdot A = A$$

$$A + A = A$$

7. Luật kết hợp (*Associative law*):

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

$$A + (B + C) = (A + B) + C$$

8. Luật bù kép:

$$\bar{\bar{A}} = A$$

9. Luật hấp thụ (*Absorption law*):

$$A \cdot (A + B) = A$$

$$A + (A \cdot B) = A$$

ThS. GVC Tô Oai Hùng

10

10

Đại Số Boolean

10. Định luật DeMorgan:

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

ThS. GVC Tô Oai Hùng

11

11

8.2.2 Các Cổng Logic

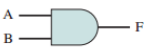
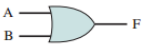
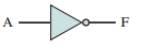
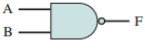
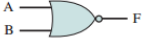
- ❖ Cổng (*gate*) là một mạch điện tử mà nó tạo tín hiệu xuất là một phép toán Boolean từ các tín hiệu vào.
- ❖ Các cổng cơ bản được sử dụng là AND, OR, NOT, NAND, NOR và XOR (Hình 2.1).
- ❖ Tất cả các cổng ngoại trừ NOT có thể có nhiều hơn hai ngõ vào.

ThS. GVC Tô Oai Hùng

12

12

Các Cổng Logic

Name	Graphical Symbol	Algebraic Function	Truth Table															
AND		$F = A \cdot B$ or $F = AB$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	0	1	0	0	1	1	1
A	B	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
OR		$F = A + B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	1
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
NOT		$F = \overline{A}$ or $F = A'$	<table><tr><th>A</th><th>F</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	F	0	1	1	0									
A	F																	
0	1																	
1	0																	
NAND		$F = \overline{AB}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
NOR		$F = \overline{A + B}$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	1	0	1	0	1	0	0	1	1	0
A	B	F																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
XOR		$F = A \oplus B$	<table><tr><th>A</th><th>B</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	F	0	0	0	0	1	1	1	0	1	1	1	0
A	B	F																
0	0	0																
0	1	1																
1	0	1																
1	1	0																

Hình 2.1: Các cổng logic cơ bản.

ThS. GVC Tô Oai Hùng

13

13

Các Cổng Logic

❖ Bất kỳ biểu thức Boolean nào cũng có thể được thực hiện bằng cách chỉ sử dụng các cổng trong mỗi tập sau:

1. AND, OR, NOT
2. AND, NOT
3. OR, NOT
4. NAND
5. NOR

❖ Ví dụ: Phép toán OR có thể được tạo ra trong tập thứ hai bằng định lý DeMorgan:

$$A + B = \overline{\overline{A} \cdot \overline{B}}$$

$$A \text{ OR } B = \text{NOT} ((\text{NOT } A) \text{ AND } (\text{NOT } B))$$

ThS. GVC Tô Oai Hùng

14

14

Các Cổng Logic

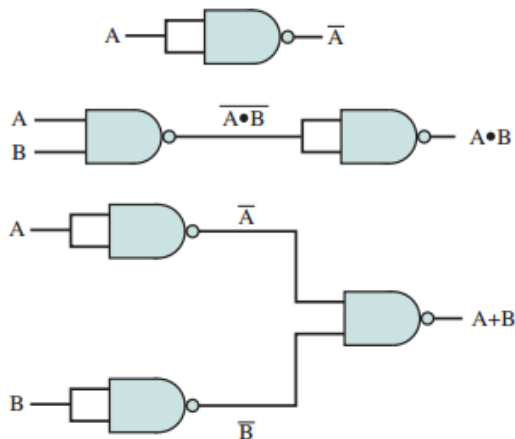
- ❖ Các cổng AND, OR, NOT có thể được tạo bởi các cổng NAND trong tập thứ 4 (Hình 2.2) hoặc các cổng NOR.
- ❖ Do vậy, các mạch số thường chỉ sử dụng các cổng NAND hay NOR.
- ❖ Bài tập tại lớp:
 - Hãy sinh phép toán AND từ phép toán OR và NOT trong tập thứ 3.
 - Sử dụng các cổng NOR trong tập thứ 5 để tạo ra AND, OR, NOT.

ThS. GVC Tô Oai Hùng

15

15

Các Cổng Logic



Hình 2.2: Sử dụng các cổng NAND để tạo ra AND, OR, NOT.

ThS. GVC Tô Oai Hùng

16

16

Bản Đồ Karnaugh

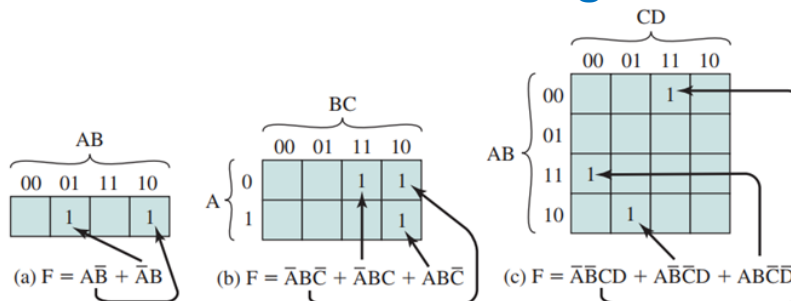
- ❖ Bản đồ Karnaugh (*Karnaugh map*) được sử dụng để đơn giản hoá biểu thức Boolean với số biến nhỏ.
- ❖ Bản đồ là một ma trận có 2^n ô vuông, thể hiện cho tất cả tổ hợp giá trị có thể của n biến nhị phân.
- ❖ Hình 2.3a cho thấy bản đồ được sử dụng cho biểu thức Boolean có hai biến. Bởi vì tổ hợp các giá trị của hai biến nhị phân A và B là: 00 ($\bar{A}\bar{B}$), 01 ($\bar{A}B$), 11 (AB) và 10 ($A\bar{B}$).
- ❖ Hình 2.4b cho thấy bản đồ của ba biến và Hình 2.4c là bốn biến.

ThS. GVC Tô Oai Hùng

17

17

Bản Đồ Karnaugh



Hình 2.3: Sử dụng bảng đồ Karnaugh để biểu diễn biểu thức Boolean.

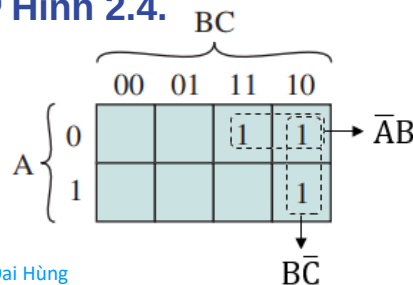
ThS. GVC Tô Oai Hùng

18

18

Bản Đồ Karnaugh

- ❖ Với mỗi toán hạng (là một tích) của một tổng, ô tương ứng trong bản đồ ghi là 1.
- ❖ Sau khi ghi vào bản đồ, nhóm các ô có giá trị 1 kề nhau trong cùng hàng hoặc cùng cột và chỉ giữ lại biến chung, đó là kết quả tối giản.
- ❖ Ví dụ, nhóm các ô kề nhau trên Hình 2.3b như Hình 2.4.



Hình 2.4: Nhóm các ô trên cùng hàng hoặc cùng cột.

ThS. GVC Tô Oai Hùng

19

19

Bản Đồ Karnaugh

- ❖ Vậy biểu thức:

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}BC + A\bar{B}\bar{C} = \bar{A}\bar{B} + \bar{B}\bar{C}$$

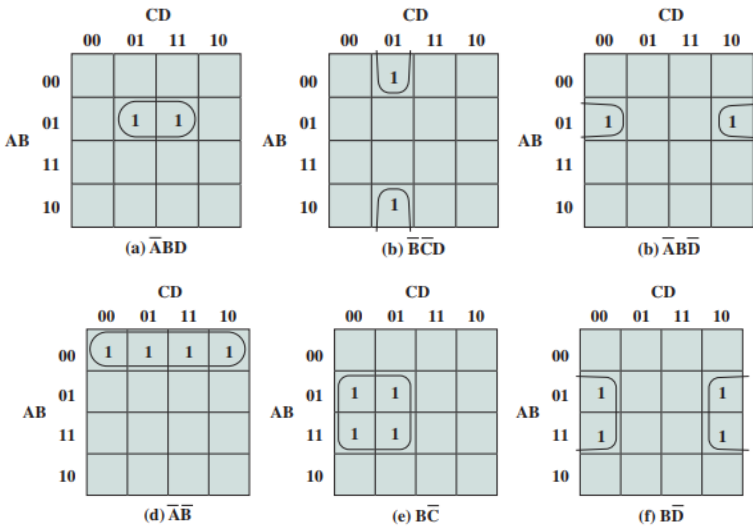
- ❖ Bài tập tại lớp: Chứng minh kết quả của biểu thức này bằng đại số Boolean.
- ❖ Hình 2.5 cho thấy một số ví dụ về nhóm các ô kề nhau trong bản đồ Karnaugh.

ThS. GVC Tô Oai Hùng

20

20

Bản Đồ Karnaugh

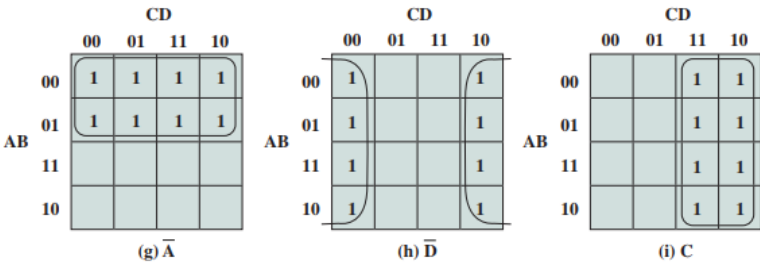


ThS. GVC Tô Oai Hùng

21

21

Bản Đồ Karnaugh



Hình 2.5: Nhóm các ô kề nhau trong bản đồ Karnaugh.

ThS. GVC Tô Oai Hùng

22

22

8.3 Mạch Tổ Hợp

- ❖ Mạch số được chia làm hai loại: Mạch tổ hợp (*combinational circuit*) và mạch tuần tự (*sequential circuit*).
- ❖ Trong mạch tổ hợp, khi các ngõ vào thay đổi trạng thái, trạng thái các ngõ ra thay đổi ngay lập tức (sau một khoảng thời gian trì hoãn rất ngắn).
- ❖ Mạch tổ hợp bao gồm n ngõ vào và m ngõ ra.
- ❖ Các mạch cộng và trừ được trình bày sau đây là các mạch tổ hợp.

ThS. GVC Tô Oai Hùng

23

23

8.3.1 Mạch Cộng Bán Phần

- ❖ Mạch cộng bán phần (*half adder* - HA) là mạch cộng 1 bit không có nhớ.
- ❖ Mạch cộng này có hai ngõ vào và hai ngõ ra.
- ❖ Xây dựng mạch này như sau:
 - Gọi A, B là các số nhị phân 1 bit cần cộng, S là bit tổng và C là bit nhớ. Ta có:

A	0	0	1	1
	+	+	+	+
B	0	1	0	1
S	0	1	1	0
C	0	0	0	1

ThS. GVC Tô Oai Hùng

24

24

Mạch Cộng Bán Phần

■ Bảng sự thật:

Single-Bit Addition			
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

■ Từ bảng sự thật, hệ thức và mạch logic như sau:

$$S = \bar{A}B + A\bar{B} = A \oplus B$$

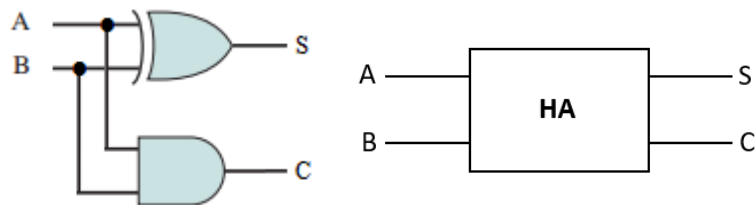
$$C = AB$$

ThS. GVC Tô Oai Hùng

25

25

Mạch Cộng Bán Phần



Hình 2.6: Mạch cộng bán phần.

ThS. GVC Tô Oai Hùng

26

26

Mạch Cộng Toàn Phần

- ❖ Để xây dựng mạch cộng nhiều bit, trước hết phải xây dựng mạch cộng 1 bit có nhớ được gọi là mạch cộng toàn phần (*full adder* - FA).
- ❖ Mạch cộng này có ba ngõ vào và hai ngõ ra.
- ❖ Mạch cộng toàn phần sẽ tính:
 - $S = A + B + C_0$
 - C là bit nhớ của phép tính tổng S .
 - C_0 là bit nhớ của phép cộng trước đó.

ThS. GVC Tô Oai Hùng

27

27

Mạch Cộng Toàn Phần

■ Bảng sự thật:

Addition with Carry Input				
C_0	A	B	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

ThS. GVC Tô Oai Hùng

28

28

Mạch Cộng Toàn Phần

- Từ bảng sự thật, để có được biểu thức logic tối giản của s và c , sử dụng một trong hai cách sau:
 - Cách 1: Sử dụng đại số Boolean.

$$S = C_0 \oplus A \oplus B$$

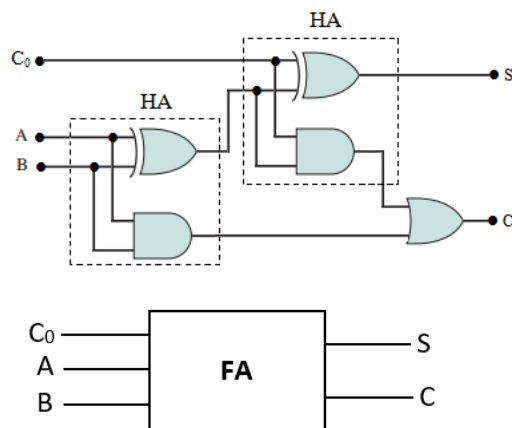
$$C = AB + C_0 (A \oplus B)$$
 - Cách 2: Sử dụng bản đồ Karnaugh.
- Bài tập tại lớp: Hãy chứng minh biểu thức của cách 1 và thực hiện cách 2 để cho thấy chúng có cùng một kết quả.

ThS. GVC Tô Oai Hùng

29

29

Mạch Cộng Toàn Phần



Hình 2.7: Mạch cộng toàn phần 1 bit.

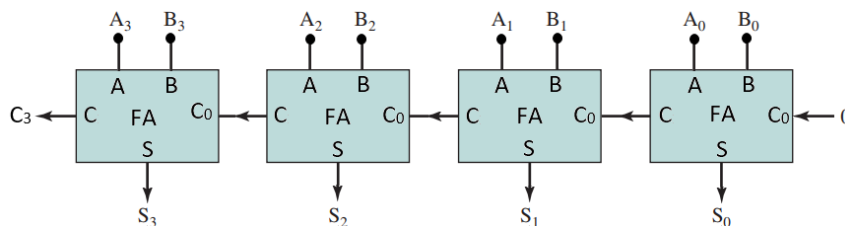
ThS. GVC Tô Oai Hùng

30

30

Mạch Cộng Nhiều Bit

- ❖ Trong phép cộng nhiều bit, phép cộng của bit LSB có thể sử dụng mạch cộng bán phần hoặc mạch cộng toàn phần với ngõ vào bit nhớ $C_0 = 0$.
- ❖ Phép cộng của các bit cao hơn phải là mạch cộng toàn phần.



Hình 2.8: Sơ đồ khối của mạch cộng 4 bit.

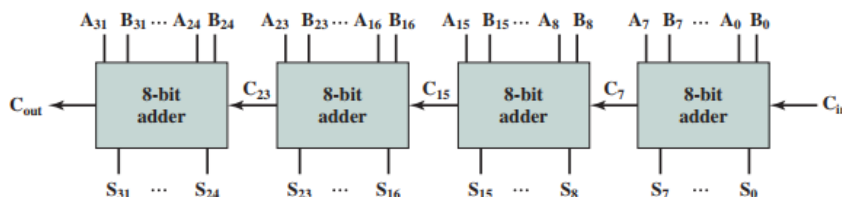
ThS. GVC Tô Oai Hùng

31

31

Mạch Cộng Nhiều Bit

- ❖ Để tạo mạch cộng nhiều bit, ghép nhiều mạch cộng lại với nhau. Hình 2.9 là sơ đồ khối của mạch cộng 32 bit.



Hình 2.9: Sơ đồ khối của mạch cộng 32 bit.

ThS. GVC Tô Oai Hùng

32

32

Một Số IC của Mạch Cộng

- ❖ 7480 : Cộng toàn phần 1 bit.
- ❖ 7482 : Cộng toàn phần 2 bit.
- ❖ 7483/LS83/283 : Cộng toàn phần 4 bit.
- ❖ 74LS183 : Hai cộng toàn phần 1 bit.

ThS. GVC Tô Oai Hùng

33

33

8.3.2 Mạch Trừ Bán Phần

- ❖ Mạch trừ bán phần (*half subtractor* - HS) là mạch trừ 1 bit không có mượn.

- Gọi A, B là các số nhị phân 1 bit cần thực hiện phép trừ, D (*difference*) là bit hiệu và Br (*borrow*) là bit mượn. Ta có:

A	0	0	1	1
	<u> </u>	<u> </u>	<u> </u>	<u> </u>
B	0	1	0	1
	<u> </u>	<u> </u>	<u> </u>	<u> </u>
D	0	1	1	0
Br	0	1	0	0

- Bảng sự thật:

A	B	D	Br
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

ThS. GVC Tô Oai Hùng

34

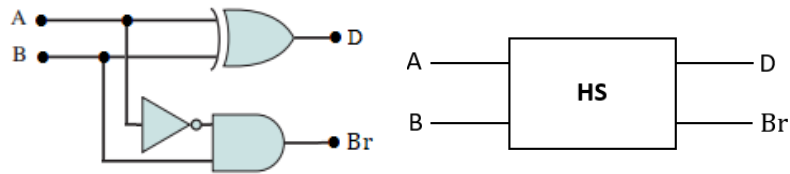
34

Mạch Trừ Bán Phần

- Từ bảng sự thật, hệ thức và mạch logic như sau:

$$D = A \oplus B$$

$$Br = \bar{A}B$$



Hình 2.10: Mạch trừ bán phần.

ThS. GVC Tô Oai Hùng

35

35

Mạch Trừ Toàn Phần

- Mạch trừ 1 bit có mượn được gọi là mạch trừ toàn phần (*full subtractor* - FS).
- Mạch trừ toàn phần sẽ tính:
 - $D = A - (B + Br_0)$.
 - Br là bit mượn của phép trừ D .
 - Br_0 là bit mượn của phép trừ trước đó.

ThS. GVC Tô Oai Hùng

36

36

Mạch Trừ Toàn Phần

■ Bảng sự thật:

Br ₀	A	B	D	Br
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	0	0
1	0	0	1	1
1	0	1	0	1
1	1	0	0	0
1	1	1	1	1

■ Hệ thức và mạch logic:

$$D = Br_0 \oplus A \oplus B$$

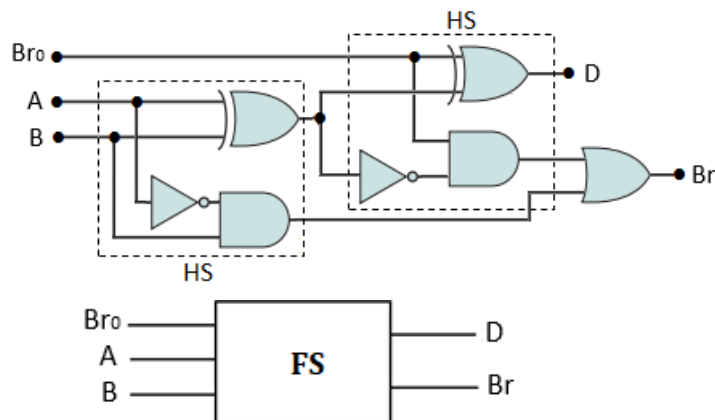
$$Br = \overline{A}B + Br_0(A \oplus B)$$

ThS. GVC Tô Oai Hùng

37

37

Mạch Trừ Toàn Phần



Hình 2.11: Mạch trừ toàn phần 1 bit.

ThS. GVC Tô Oai Hùng

38

38

8.4 Mạch Tuần Tụ

- ❖ Trong mạch tuần tụ, trạng thái các ngõ ra không những phụ thuộc vào trạng thái các ngõ vào mà còn phụ thuộc vào trạng thái trước đó của các ngõ ra.
- ❖ Mạch tuần tụ có tính nhớ và tính đồng bộ.

ThS. GVC Tô Oai Hùng

39

39

Mạch Chốt RS và Flip-Flop RS

- ❖ Dạng đơn giản nhất của mạch tuần tụ là mạch chốt (mạch cài) và flip-flop (mạch lật).
- ❖ Có nhiều loại mạch chốt và flip-flop, chúng có hai thuộc tính:
 - Có hai trạng thái ổn định và duy trì trạng thái đó khi trạng thái ngõ vào chưa thay đổi. Do đó, chúng có chức năng như là 1 bit nhớ.
 - Có hai ngõ ra có giá trị ngược nhau, thường được gọi là Q và $\bar{Q}$. Kết quả được tính theo ngõ Q .

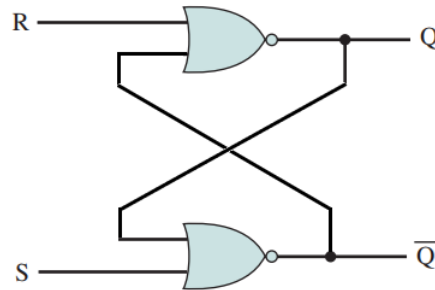
ThS. GVC Tô Oai Hùng

40

40

Mạch Chốt RS

- ❖ Hình 2.12 là mạch chốt RS (*RS latch*) với hai cổng NOR được mắc hồi tiếp với nhau (có thể dùng cổng NAND).
- ❖ Mạch này có hai ngõ vào là R (*reset*) và S (*set*), hai ngõ ra là Q và $\bar{Q}$.



Hình 2.12: Mạch chốt RS.

ThS. GVC Tô Oai Hùng

41

41

Mạch Chốt RS

- ❖ Trong bảng 2.2, các ngõ vào $S = 1$, $R = 1$ thì không được phép.

Input		Output
S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Cấm

Bảng 2.2: Bảng sự thật của mạch chốt RS.

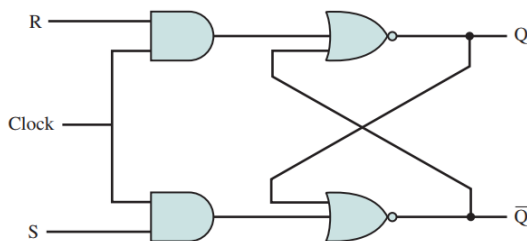
ThS. GVC Tô Oai Hùng

42

42

Flip-Flop RS

- ❖ Để kiểm soát ngõ ra và tạo sự đồng bộ, mạch flip-flop RS được sử dụng thay cho chốt RS.



Hình 2.13: Flip-flop RS.

Input			Output
S	R	CLK	Q_{n+1}
0	0	↑	Q_n
0	1	↑	0
1	0	↑	1
1	1	↑	Cấm

Bảng 2.3: Bảng sự thật của flip-flop RS.

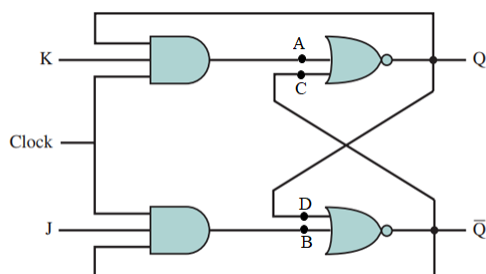
ThS. GVC Tô Oai Hùng

43

43

Flip-Flop JK

- ❖ Flip-flop JK (Jack Kilby – kỹ sư điện tử người Mỹ, tạo ra mạch lật vào năm 1958) khắc phục được trạng thái cấm của flip-flop RS.



Hình 2.14: Flip-flop JK.

Input			Output
J	K	CLK	Q_{n+1}
0	0	↑	Q_n
0	1	↑	0
1	0	↑	1
1	1	↑	$\overline{Q_n}$

Bảng 2.4: Bảng sự thật của flip-flop JK.

ThS. GVC Tô Oai Hùng

44

44

Flip-Flop JK

❖ Lưu ý:

- Giả sử $Q = 0$, $\bar{Q} = 1$ và gọi Δt là thời gian dữ liệu truyền qua một cổng.
- Khi $J = K = 1$ thì:
 - Δt thứ 1: $A = 0$, $B = 1$.
 - Δt thứ 2: $Q = 1$.
 - Δt thứ 3: $\bar{Q} = 0$.
- Vậy, sau $3\Delta t$ ngõ ra $Q = 1$, $\bar{Q} = 0$ ($Q_{n+1} = \bar{Q}_n$).
- Sau $3\Delta t$ nữa, nếu clock còn ở mức cao thì sự hồi tiếp trở về của Q và $\bar{Q}$ sẽ làm cho chúng có giá trị khác là $Q = 0$ và $\bar{Q} = 1$, quá trình trên lặp lại.

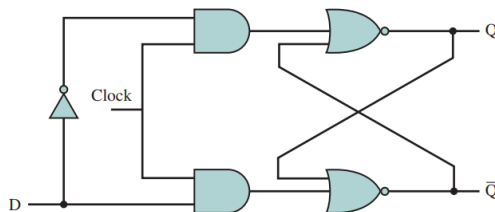
ThS. GVC Tô Oai Hùng

45

45

Flip-Flop D

- ❖ Khi nối các ngõ vào của flip-flop RS (hoặc JK), chúng ta được flip-flop D.
- ❖ Ngõ ra của flip-flop D luôn bằng với giá trị đã gán trước khi có xung nhịp cho ngõ vào.



Hình 2.15: Flip-flop D.

Input		Output
D	CLK	Q_{n+1}
0	↑	0
1	↑	1

Bảng 2.5: Bảng sự thật của flip-flop D.

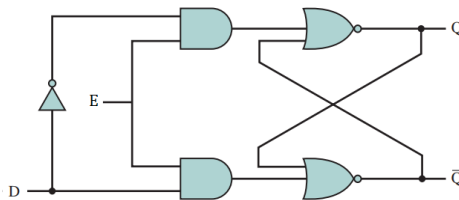
ThS. GVC Tô Oai Hùng

46

46

Chốt D

- ❖ Trong flip-flop D, nếu thay ngõ vào đồng hồ bằng ngõ vào cho phép E (*enable*) tác động ở mức cao, chúng ta được mạch chốt D.



Hình 2.16: Chốt D.

Input		Output
D	E	Q_{n+1}
0	1	0
1	1	1

Bảng 2.6: Bảng sự thật của chốt D.

ThS. GVC Tô Oai Hùng

47

47

Một Số IC Flip-Flop và Chốt

- ❖ 7470: Flip-flop JK kích hoạt cạnh lên.
- ❖ 7473: Flip-flop JK kích hoạt cạnh xuống.
- ❖ 74HC175: 4 flip-flop D.
- ❖ 74LS375: 4 chốt D.

ThS. GVC Tô Oai Hùng

48

48

8.5 Thanh Ghi

- ❖ Thanh ghi là mạch số được dùng trong CPU để lưu trữ nhiều bit dữ liệu. Có hai loại thanh ghi thường sử dụng: Thanh ghi song song (*parallel register*) và thanh ghi dịch (*shift register*).

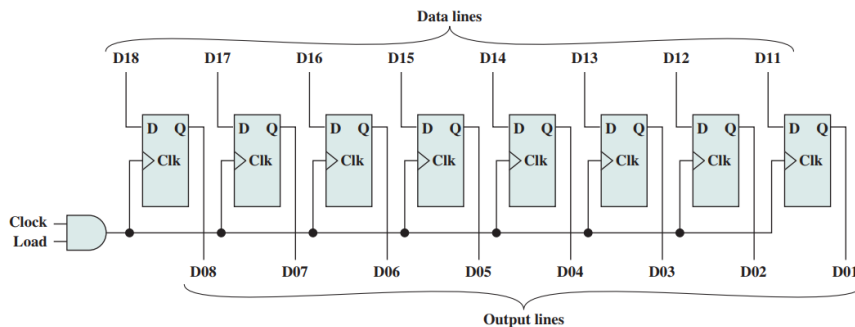
ThS. GVC Tô Oai Hùng

49

49

8.5.1 Thanh Ghi Song Song

- ❖ Thanh ghi song song chứa các bit nhớ có thể đọc hoặc ghi đồng thời.



Hình 2.17: Thanh ghi song song 8 bit.

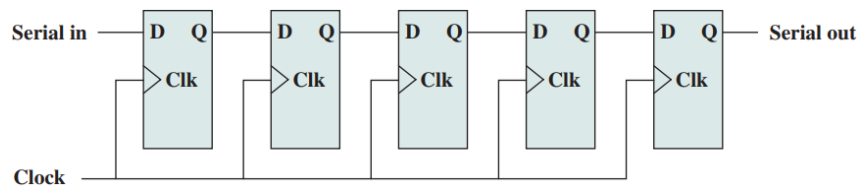
ThS. GVC Tô Oai Hùng

50

50

8.5.2 Thanh Ghi Dịch

- ❖ Thanh ghi dịch nhận hoặc chuyển thông tin tuần tự.
- ❖ Dữ liệu vào từ flip-flop bên trái nhất. Với mỗi chu kỳ xung nhịp, dữ liệu được dịch sang phải một vị trí và bit bên phải nhất được chuyển ra ngoài.



Hình 2.18: Thanh ghi dịch 5 bit.

ThS. GVC Tô Oai Hùng

51

51

8.6 Bộ Đếm

- ❖ Một công dụng khác của mạch tuần tự là bộ đếm (*counter*).
- ❖ Bộ đếm là thanh ghi, nếu nó được tạo thành từ n flip-flop thì có thể đếm từ 0 đến $2^n - 1$.
- ❖ Một ví dụ về bộ đếm trong CPU là bộ đếm chương trình.
- ❖ Bộ đếm có thể được thiết kế không đồng bộ (*asynchronous*) hoặc đồng bộ (*synchronous*).
- ❖ Bộ đếm đồng bộ nhanh hơn so với bộ đếm không đồng bộ.

ThS. GVC Tô Oai Hùng

52

52

Bộ Đếm Đồng Bộ

- ❖ Lấy ví dụ về bộ đếm đồng bộ 3 bit bằng 3 flip-flop JK.
- ❖ Trước khi xây dựng bảng sự thật cho bộ đếm, xem lại bảng sự thật của flip-flop JK:

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

- ❖ Thêm trạng thái Q_n (ngõ ra hiện tại) vào bảng, thực hiện nhóm các cặp hàng mà J hoặc K có cùng giá trị, được Bảng 2.7.

ThS. GVC Tô Oai Hùng

53

53

Bộ Đếm Đồng Bộ

Q_n	J	K	Q_{n+1}
0	0	x	0
0	1	x	1
1	x	1	0
1	x	0	1

Bảng 2.7: Bảng sự thật của flip-flop JK khi thêm Q_n .

- ❖ Lưu ý: x thể hiện cho giá trị 0 hoặc 1.
- ❖ Gọi A, B và C lần lượt là các bit của ngõ ra (ứng với Q_A , Q_B và Q_C) của 3 flip-flop A, B và C. Trong đó, C là bit cao nhất.

ThS. GVC Tô Oai Hùng

54

54

Bộ Đếm Đồng Bộ

- ❖ Ta sẽ lập bảng sự thật để mô tả mối quan hệ giữa các ngõ ra này với các ngõ vào J, K của 3 flip-flop.
- ❖ Trước tiên, xét 2 hàng của bảng:

C	B	A	Thập phân	J _C	K _C	J _B	K _B	J _A	K _A
0	0	0	0						
0	0	1	1						

- ❖ Giá trị của ngõ ra C được giữ nguyên 0, giá trị của ngõ ra B được giữ nguyên 0 và giá trị của ngõ ra A được chuyển từ 0 thành 1 khi có xung nhịp tiếp theo.

ThS. GVC Tô Oai Hùng

55

55

Bộ Đếm Đồng Bộ

- ❖ Dựa vào bảng 2.7, ta sẽ ghi giá trị của J và K tương ứng cho mỗi flip-flop như sau:

C	B	A	Thập phân	J _C	K _C	J _B	K _B	J _A	K _A
0	0	0	0	0	x	0	x	1	x
0	0	1	1						

- ❖ Bằng cách lập luận tương tự, các hàng còn lại của bảng được điền vào như Bảng 2.8.

ThS. GVC Tô Oai Hùng

56

56

Bộ Đếm Đồng Bộ

C	B	A	Thập phân	J_C	K_C	J_B	K_B	J_A	K_A
0	0	0	0	0	x	0	x	1	x
0	0	1	1	0	x	1	x	x	1
0	1	0	2	0	x	x	0	1	x
0	1	1	3	1	x	x	1	x	1
1	0	0	4	x	0	0	x	1	x
1	0	1	5	x	0	1	x	x	1
1	1	0	6	x	0	x	0	1	x
1	1	1	7	x	1	x	1	x	1
0	0	0	0						

Bảng 2.8: Bảng sự thật của bộ đếm đồng bộ 3 bit.

ThS. GVC Tô Oai Hùng

57

57

Bộ Đếm Đồng Bộ

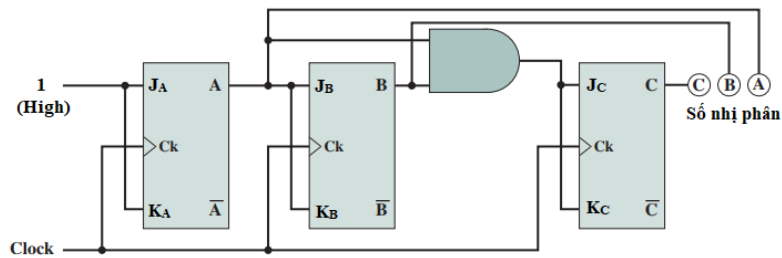
- ❖ Bảng 2.8 hiển thị các giá trị bắt buộc của tất cả ngõ vào J và K dưới dạng các hàm của giá trị các ngõ ra hiện tại là A, B và C.
- ❖ Với sự trợ giúp của bản đồ Karnaugh, ta có thể tính được các biểu thức Boolean dạng tối giản cho sáu hàm này:
 - $J_A = K_A = 1$
 - $J_B = K_B = A$
 - $J_C = K_C = BA$
- ❖ Từ các biểu thức trên, sơ đồ mạch của bộ đếm như Hình 2.19.

ThS. GVC Tô Oai Hùng

58

58

Bộ Đếm Đồng Bộ



Hình 2.19: Bộ đếm đồng bộ 3 bit.

- ❖ Lưu ý: Dựa vào giá trị của ba ngõ ra A, B và C trong Bảng 2.8, chúng ta có nhận xét sau:
 - Với flip-flop A: Ngõ ra A thay đổi trạng thái ($Q_{n+1} = \overline{Q_n}$) sau từng xung clock. Vậy, dựa vào bảng sự thật của flip-flop JK: $J_A = K_A = 1$.

ThS. GVC Tô Oai Hùng

59

59

Bộ Đếm Đồng Bộ

- Với flip-flop B: Ngõ ra B thay đổi trạng thái nếu trước đó ngõ ra A bằng 1. Vậy: $J_B = K_B = A$. Nghĩa là, khi $A = 1$ thì $J_B = K_B = A = 1$, nên $Q_{n+1} = \overline{Q_n}$ làm thay đổi trạng thái.
- Với flip-flop C: Ngõ ra C thay đổi trạng thái nếu trước đó ngõ ra A và ngõ ra B đều bằng 1. Vậy: $J_C = K_C = AB$. Nghĩa là, khi $A = B = 1$ thì $J_C = K_C = AB = 1$, nên $Q_{n+1} = \overline{Q_n}$ làm thay đổi trạng thái.
- ❖ Từ các nhận xét trên, ta cũng có được mạch như hình 2.19.

ThS. GVC Tô Oai Hùng

60

60